

Functional Programming — Advanced

Senior Developer Level

এই section-এর মূল লক্ষ্য হলো JavaScript-এ Functional Programming (FP) শুধু syntax হিসেবে না দেখে **data transformation**, **error handling**, **composition** এবং **predictable program design** হিসেবে বোঝা।

আমরা মূলত ৬টি core topic দেখব:

1. Maybe Monad — null-safe chaining
2. Either Monad — exception ছাড়াই error handling
3. Functor & `flatMap` / bind
4. Lazy Evaluation with Generators
5. Point-free Programming
6. Algebraic Data Types (ADT)

এর সাথে related concepts:

- Pure Function
- Immutability
- Referential Transparency
- Higher-Order Function
- Function Composition
- Currying
- Partial Application
- Functor
- Monad
- Option / Maybe
- Either / Result
- Sum Type
- Product Type
- Tagged Union
- Railway-oriented programming

1. Functional Programming আসলে কী?

Functional Programming হলো এমন একটি programming paradigm যেখানে program-কে মূলত **function** এবং **data transformation** হিসেবে চিন্তা করা হয়।

একটি imperative approach:

```
let total = 0;

for (const price of prices) {
  total += price;
}

console.log(total);
```

Functional approach:

```
const total = prices.reduce(
  (sum, price) => sum + price,
  0
);
```

এখানে দ্বিতীয় approach-এ আমরা বেশি focus করছি:

"কীভাবে data transform হবে?"

বরং:

"step by step কীভাবে variable পরিবর্তন করব?"

2. Functional Programming-এর Core Principles

Advanced FP বোঝার আগে কয়েকটি foundation খুব গুরুত্বপূর্ণ।

2.1 Pure Function

Pure function একই input দিলে সবসময় একই output দেয় এবং বাইরে কোনো observable side effect করে না।

```
function add(a, b) {
  return a + b;
}
```

এটা pure।

কিন্তু:

```
let total = 0;

function addToTotal(value) {
  total += value;
}
```

এটা pure না।

কারণ function-এর result বাইরের state-এর উপর নির্ভর করছে এবং বাইরের state পরিবর্তন করছে।

2.2 Referential Transparency

যদি কোনো expression-এর জায়গায় তার result বসিয়ে দিলেও program-এর behavior পরিবর্তন না হয়, তাহলে expression-টি referentially transparent।

```
const result = add(10, 20);
```

যদি `add(10, 20)` সবসময় `30` হয়, তাহলে:

```
const result = 30;
```

দিয়ে replace করা যায়।

এটাই functional programming-এর predictability-এর একটি বড় কারণ।

3. Immutability

FP-তে সাধারণত existing data mutate না করে নতুন data তৈরি করা হয়।

খারাপ:

```
user.name = "Rahim";
```

Functional style:

```
const updatedUser = {
  ...user,
```

```
    name: "Rahim"
  };
```

Array-এর ক্ষেত্রেও:

```
const updated = users.map(user =>
  user.id === 10
    ? { ...user, active: true }
    : user
);
```

এতে original `users` পরিবর্তন হচ্ছে না।

4. Higher-Order Function

যে function অন্য function নেয় অথবা function return করে তাকে Higher-Order Function বলে।

```
function multiplyBy(factor) {
  return function (value) {
    return value * factor;
  };
}

const double = multiplyBy(2);

double(10);
```

Result:

20

`map`, `filter`, `reduce` নিজেরাও Higher-Order Function-এর বড় উদাহরণ।

5. Function Composition

ছোট ছোট function combine করে বড় behavior তৈরি করা।

ধরি:

```
const trim = value => value.trim();

const toLower = value => value.toLowerCase();

const addPrefix = value => `user:${value}`;
```

তারপর:

```
const result = addPrefix(
  toLower(
    trim(" SHARIAR ")
  )
);
```

Result:

```
user:shariar
```

Composition-এর লক্ষ্য:

```
input
↓
trim
↓
toLower
↓
addPrefix
↓
output
```

এই concept পরে Maybe/Either এবং point-free programming-এ অনেক কাজে আসবে।

6. Currying

একটি function-এর multiple arguments-কে একবারে না নিয়ে একেকটি করে নেওয়াকে currying বলা হয়।

```
const add = a => b => a + b;

const add10 = add(10);
```

```
add10(5);
```

Result:

```
15
```

এখানে:

```
add(10)
```

একটি নতুন function তৈরি করেছে।

7. Maybe Monad — Null-safe Chaining

এখন আসি প্রথম মূল topic-এ।

ধরি আমাদের কাছে nested object:

```
const user = {  
  profile: {  
    address: {  
      city: "Rangpur"  
    }  
  }  
};
```

আমরা city access করতে চাই।

```
user.profile.address.city
```

সমস্যা হলো কোনো একটি property missing হলে:

```
user.profile.address
```

যদি `undefined` হয় তাহলে:

```
user.profile.address.city
```

crash করবে।

8. Traditional Null Checking

আমরা করতে পারি:

```
if (
  user &&
  user.profile &&
  user.profile.address
) {
  console.log(user.profile.address.city);
}
```

অনেক nested হলে code ugly হয়ে যায়।

Modern JavaScript-এ:

```
user?.profile?.address?.city
```

এটা অনেক ভালো।

কিন্তু FP-তে আমরা এই problem-টাকে আরও abstractভাবে ভাবতে পারি।

9. Maybe / Option কী?

Maybe-এর basic idea:

"Value থাকতে পারে, আবার নাও থাকতে পারে।"

দুটি state:

```
Some(value)
None
```

অথবা:

```
Just(value)
Nothing
```

JavaScript-এ built-in Maybe type নেই।

তাই আমরা নিজে model করতে পারি।

```
const Some = value => ({
  type: "Some",
  value
});

const None = () => ({
  type: "None"
});
```

10. Maybe-এর Basic Idea

```
const user = Some({
  name: "Sharir"
});
```

এখানে value আছে।

অন্যদিকে:

```
const user = None();
```

মানে value নেই।

এখন আমরা সরাসরি value নিয়ে কাজ না করে container-এর মাধ্যমে কাজ করি।

11. Maybe-এর map

ধরি:

```
const Some = value => ({
  type: "Some",
  value
});

const None = () => ({
  type: "None"
});
```

একটি helper:

```
function mapMaybe(maybe, fn) {
  if (maybe.type === "None") {
    return maybe;
  }

  return Some(fn(maybe.value));
}
```

এখন:

```
const value = Some(10);

const result = mapMaybe(
  value,
  x => x * 2
);
```

Result:

Some(20)

কিন্তু:

```
const value = None();

const result = mapMaybe(
  value,
  x => x * 2
);
```

Result:

None

Function execute-ই হবে না।

12. কেন এটা useful?

ধরি:

```
const getProfile = user => user.profile;

const getAddress = profile => profile.address;

const getCity = address => address.city;
```

Normal code:

```
getCity(
  getAddress(
    getProfile(user)
  )
);
```

কিন্তু মাঝখানে `null` এলে সমস্যা।

Maybe abstraction বলছে:

Value থাকলে → next transformation
Value না থাকলে → None carry forward

এটাই null-safe pipeline।

13. Maybe Chain

একটি practical implementation:

```
class Maybe {
  constructor(value) {
```

```

    this.value = value;
  }

  static of(value) {
    return new Maybe(value);
  }

  map(fn) {
    if (this.value == null) {
      return this;
    }

    return Maybe.of(fn(this.value));
  }

  getOrElse(defaultValue) {
    return this.value ?? defaultValue;
  }
}

```

ব্যবহার:

```

const city = Maybe
  .of(user)
  .map(user => user.profile)
  .map(profile => profile.address)
  .map(address => address.city)
  .getOrElse("Unknown");

```

যদি কোনো step-এ `null` বা `undefined` আসে:

```

Maybe(user)
  ↓
Maybe(profile)
  ↓
Maybe(address)
  ↓
Maybe(city)

```

আর value না থাকলে chain safe থাকে।

14. Maybe Monad কেন "Monad"?

এখানে একটি গুরুত্বপূর্ণ distinction:

`map()` শুধু value transform করে।

```
Maybe(A)
  ↓ map
Maybe(B)
```

কিন্তু যদি function নিজেই Maybe return করে:

```
findUser(id)
```

return:

```
Maybe(User)
```

তখন:

```
Maybe(A)
  ↓ map
Maybe(Maybe(B))
```

nested structure তৈরি হয়।

এটা flatten করতে দরকার:

```
flatMap
```

15. Either Monad

Maybe মূলত বলেছে:

```
Value আছে / নেই
```

কিন্তু অনেক সময় আমাদের আরও information দরকার।

ধরি API call:

```
createUser(data)
```

failure হতে পারে:

```
Invalid email  
Email already exists  
Database unavailable  
Permission denied
```

শুধু:

```
null
```

দিলে `error`-এর কারণ হারিয়ে যাবে।

এখানে `Either` useful।

16. Either-এর দুই Side

`Either`-এর সাধারণ দুই constructor:

```
Left  
Right
```

Convention:

```
Left  = failure  
Right = success
```

অর্থাৎ:

```
Either<Error, Value>
```

17. Simple Either Implementation

```
const Left = value => ({
  type: "Left",
  value
});

const Right = value => ({
  type: "Right",
  value
});
```

Success:

```
Right({
  id: 10,
  name: "Sharlar"
});
```

Failure:

```
Left("User not found");
```

18. Either-এর map

```
function mapEither(result, fn) {
  if (result.type === "Left") {
    return result;
  }

  return Right(fn(result.value));
}
```

Success:

```
mapEither(
  Right(10),
```

```
x => x * 2
);
```

Result:

```
Right(20)
```

Failure:

```
mapEither(
  Left("Database error"),
  x => x * 2
);
```

Result:

```
Left("Database error")
```

Function execute হবে না।

19. Railway-Oriented Programming

Either-এর সবচেয়ে সুন্দর mental model:

```

      ┌ failure → Left → Left → Left
      |
Input → validate → transform → save → response
      |
      └ success → Right → Right → Right
```

একবার failure track-এ গেলে পরের successful transformations আর execute হয় না।

এটাকে অনেক সময় **Railway-Oriented Programming** বলা হয়।

20. Example: User Registration

ধরি:

```
validateUser  
normalizeUser  
saveUser
```

প্রতিটি function:

```
Either<Error, User>
```

return করে।

Pipeline:

```
validateUser(data)  
  .flatMap(normalizeUser)  
  .flatMap(saveUser)
```

যদি validation fail করে:

```
Left("Invalid email")
```

তাহলে:

```
normalizeUser
```

আর চলবে না।

21. map বনাম flatMap

এটি Advanced FP-এর সবচেয়ে গুরুত্বপূর্ণ distinction-এর একটি।

ধরি:

```
const double = x => x * 2;
```

এটা normal value return করে:

```
10 → 20
```

তাই:

```
Maybe(10).map(double)
```

হবে:

```
Maybe(20)
```

এখন:

```
const findUser = id => Maybe(User);
```

Function already Maybe return করছে।

তখন:

```
Maybe(10).map(findUser)
```

হতে পারে:

```
Maybe(Maybe(User))
```

Nested container।

তাই দরকার:

```
Maybe(10).flatMap(findUser)
```

Result:

```
Maybe(User)
```

22. Functor কী?

Functor হলো এমন একটি structure যার মধ্যে থাকা value-কে `map` করা যায়।

Conceptually:

`F<A>`

থেকে:

`F<B>`

যেতে পারি:

`map : (A → B) → F<A> → F<B>`

উদাহরণ:

`Array<A>`
↓ `map`
`Array<B>`

`[1, 2, 3].map(x => x * 2);`

Result:

`[2, 4, 6]`

`Array` এখানে `functor`-এর মতো আচরণ করছে।

23. Maybe-ও Functor-এর মতো

`Maybe(10)`

এর মধ্যে:

```
x => x * 2
```

apply করলে:

```
Maybe (20)
```

অর্থাৎ container-এর structure intact রেখে ভিতরের value transform হচ্ছে।

24. Functor-এর গুরুত্বপূর্ণ ধারণা: Structure Preserve

Functor সাধারণত:

```
container  
+  
map
```

এই idea-কে represent করে।

Array:

```
[1, 2, 3]
```

map:

```
[2, 4, 6]
```

Maybe:

```
Some (10)
```

map:

```
Some (20)
```

Either:

```
Right(10)
```

map:

```
Right(20)
```

Structure preserve হয়েছে।

25. Functor Laws

Advanced level-এ Functor-এর কিছু mathematical law আছে।

Identity

```
map(identity) = identity
```

অর্থাৎ:

```
value.map(x => x)
```

এর ফলে structure-এর কোনো পরিবর্তন হওয়া উচিত নয়।

Composition

```
map(f).map(g)
```

conceptually equivalent:

```
map(x => g(f(x)))
```

অর্থাৎ:

```
container
  .map(f)
  .map(g);
```

এবং:

```
container.map(
  x => g(f(x))
);
```

একই logical result দেওয়া উচিত।

26. Monad কী?

Monad-কে খুব সহজভাবে মনে রাখার একটি practical definition:

Monad এমন একটি abstraction যা context-এর ভিতরের value নিয়ে এমন function chain করতে দেয় যেখানে function নিজেই একই context-এর value return করতে পারে।

ধরি:

$$A \rightarrow M\langle B \rangle$$

এবং:

$$M\langle A \rangle$$

থেকে আমরা:

$$M\langle B \rangle$$

এ যেতে চাই।

তখন:

```
flatMap
```

ব্যবহার করি।

27. Monad-এর মূল সমস্যা

ধরি:

```
const getUser = id => Maybe(User);  
  
const getProfile = user => Maybe(Profile);  
  
const getAddress = profile => Maybe(Address);
```

এখন:

```
Maybe(id)
```

থেকে chain:

```
Maybe(ID)  
  ↓  
Maybe(User)  
  ↓  
Maybe(Profile)  
  ↓  
Maybe(Address)
```

প্রতিটি function Maybe return করছে।

`flatMap` nested Maybe তৈরি হতে দেয় না।

28. Monad-এর দুই গুরুত্বপূর্ণ operation

Conceptually:

```
of / pure
```

এবং:

flatMap / bind

of :

$A \rightarrow M<A>$

flatMap :

$M<A> + (A \rightarrow M) \rightarrow M$

29. Maybe বনাম Either

Maybe	Either
Value আছে/নেই	Success/failure
None	Left(error)
Some(value)	Right(value)
Error detail সাধারণত নেই	Error detail রাখা যায়
Optional data	Explicit failure
Null-safe access	Validation/business errors

সহজভাবে:

Maybe = "কিছু আছে কি?"

Either = "কাজটা সফল হয়েছে কি? না হলে কেন?"

30. Lazy Evaluation

এখন আসি Lazy Evaluation-এ।

Normally JavaScript expression eager।

```
const result = expensiveFunction();
```

`result` তৈরি করার সময়ই function execute হবে।

Lazy evaluation-এর idea:

প্রয়োজন না হওয়া পর্যন্ত computation করো না।

31. Eager vs Lazy

ধরি:

```
const numbers = [1, 2, 3, 4, 5];
```

Eager:

```
const result = numbers
  .map(x => x * 2)
  .filter(x => x > 5);
```

প্রথমে পুরো map:

```
[2,4,6,8,10]
```

তারপর filter:

```
[6,8,10]
```

অর্থাৎ intermediate array তৈরি হচ্ছে।

32. Generator দিয়ে Lazy Evaluation

Generator এখানে খুব useful।

```
function* numbers() {
  let i = 1;
```

```
while (true) {  
  yield i++;  
}
```

এটা infinite sequence ।

```
const nums = numbers();
```

এখানে infinite loop execute হয়ে যায়নি ।

কারণ generator lazy ।

33. yield

```
function* numbers() {  
  yield 1;  
  yield 2;  
  yield 3;  
}
```

```
const iterator = numbers();  
  
iterator.next();
```

Result:

```
{  
  value: 1,  
  done: false  
}
```

পরের:

```
iterator.next();
```

Result:

```
{  
  value: 2,  
  done: false  
}
```

অর্থাৎ computation demand অনুযায়ী হচ্ছে।

34. Lazy Pipeline

আমরা generator দিয়ে lazy transformation বানাতে পারি।

```
function* map(iterable, fn) {  
  for (const value of iterable) {  
    yield fn(value);  
  }  
}
```

Filter:

```
function* filter(iterable, predicate) {  
  for (const value of iterable) {  
    if (predicate(value)) {  
      yield value;  
    }  
  }  
}
```

ব্যবহার:

```
const source = [1, 2, 3, 4, 5];  
  
const doubled = map(  
  source,  
  x => x * 2  
);  
  
const filtered = filter(  
  doubled,  
  x => x > 5  
);
```

এখনও পুরো pipeline execute হয়নি।

35. Demand-driven Execution

যখন:

```
filtered.next();
```

চাওয়া হবে, তখন computation শুরু হবে।

Mental model:

```
source
  ↓
map
  ↓
filter
  ↓
next()
```

`next()` হলো demand।

36. Lazy Evaluation-এর বড় সুবিধা

ধরি ১ কোটি item:

```
const hugeData = ...
```

Eager pipeline:

```
source
  ↓
map → intermediate array
  ↓
filter → another array
  ↓
take
```

অনেক memory লাগতে পারে।

Lazy:

```
source → map → filter → take
                        ↑
                    demand
```

শুধু যতটুকু দরকার ততটুকুই process হতে পারে।

37. Infinite Data Structure

Lazy evaluation-এর সবচেয়ে সুন্দর example:

```
function* infiniteNumbers() {
  let n = 1;

  while (true) {
    yield n++;
  }
}
```

এখানে theoretically infinite data।

কিন্তু আমরা:

```
const numbers = infiniteNumbers();

numbers.next();
numbers.next();
numbers.next();
```

করে প্রয়োজনমতো নিতে পারি।

38. Lazy Evaluation কোথায় Useful?

বিশেষ করে:

- Large datasets

- Streaming
- Pagination
- Infinite sequences
- File processing
- Data pipelines
- ETL
- Log processing
- Memory-sensitive applications

Node.js streams-এর সাথে এই idea-এর strong conceptual connection আছে।

39. Point-free Programming

Point-free style হলো function-এর argument explicitly না লিখে function composition-এর মাধ্যমে logic প্রকাশ করা।

Normal:

```
const double = x => x * 2;  
  
const numbers = [1, 2, 3];  
  
const result = numbers.map(  
  x => double(x)  
);
```

Point-free-ish:

```
const result = numbers.map(double);
```

এখানে:

```
x
```

explicitly লেখা হয়নি।

40. Point-free আরও ভালোভাবে বোঝা

ধরি:

```
const toUpper = str => str.toUpperCase();

const trim = str => str.trim();
```

Normal:

```
const normalize = value =>
  toUpper(trim(value));
```

Composition দিয়ে:

```
const compose = (f, g) =>
  x => f(g(x));

const normalize =
  compose(toUpper, trim);
```

এখানে `value` নেই।

এটাই point-free style-এর মূল idea।

41. Point-free-এর সুবিধা

1. Transformation পরিকার হয়

```
trim → lowercase → prefix
```

এরকম pipeline দেখা যায়।

2. Reusable functions

ছোট functionগুলো আলাদা করে ব্যবহার করা যায়।

3. Composition সহজ হয়

```
const process =
  compose(
    save,
    validate,
```

```
normalize  
);
```

42. Point-free-এর সমস্যা

সব জায়গায় point-free ভালো না।

অতিরিক্ত করলে:

```
const f = compose(  
  map(transform),  
  filter(predicate),  
  reduce(combine)  
);
```

নতুন developer-এর কাছে কঠিন হতে পারে।

তাই principle:

Point-free readability বাড়ালে ব্যবহার করো, readability কমালে ব্যবহার করো না।

Functional Programming মানেই সবকিছু point-free করা নয়।

43. Algebraic Data Types (ADT)

এটা Advanced FP-এর খুব গুরুত্বপূর্ণ concept।

ADT মূলত data structure-কে composition-এর মাধ্যমে model করার একটি mathematical idea।

দুইটি প্রধান category:

```
Product Type  
Sum Type
```

44. Product Type

Product type মানে:

একসাথে একাধিক value থাকবে।

JavaScript object হলো এর practical example।

```
const user = {  
  id: 10,  
  name: "Sharir",  
  age: 31  
};
```

এখানে:

User = ID × Name × Age

Conceptually সব field একসাথে আছে।

45. Sum Type

Sum type মানে:

অনেক possible variant-এর মধ্যে একটি variant থাকবে।

ধরি payment status:

Pending
Success
Failed

JavaScript-এ:

```
const payment =  
  { type: "pending" };
```

অথবা:

```
const payment =  
  {  
    type: "success",
```

```
    transactionId: "TX123"  
  };
```

অথবা:

```
const payment =  
  {  
    type: "failed",  
    reason: "Insufficient balance"  
  };
```

একটি সময় একটাই variant ।

46. Tagged Union / Discriminated Union

JavaScript-এ Sum Type model করার সবচেয়ে practical pattern হলো **discriminated union** ।

```
const result = {  
  type: "success",  
  data: {  
    id: 10  
  }  
};
```

অথবা:

```
const result = {  
  type: "error",  
  message: "User not found"  
};
```

এখানে:

```
type
```

হলো discriminator/tag ।

47. ADT দিয়ে API Response Model

ধরি API তিন ধরনের response দেয়:

```
Loading  
Success  
Error
```

আমরা model করতে পারি:

```
const loading = {  
  type: "loading"  
};  
  
const success = {  
  type: "success",  
  data: users  
};  
  
const error = {  
  type: "error",  
  message: "Failed to load users"  
};
```

এখন UI:

```
function render(state) {  
  switch (state.type) {  
    case "loading":  
      return "Loading...";  
  
    case "success":  
      return state.data;  
  
    case "error":  
      return state.message;  
  }  
}
```

এটা React state management-এও খুব useful pattern ।

48. Product + Sum একসাথে

ধরি:

```
Payment =  
  Pending  
  | Success(TransactionId)  
  | Failed(Error)
```

এটা Sum Type।

কিন্তু:

```
Success(TransactionId)
```

নিজেই Product Type হতে পারে যদি একাধিক field থাকে:

```
{  
  type: "success",  
  transactionId: "TX123",  
  amount: 500,  
  timestamp: 123456  
}
```

তাই complex data model:

```
Sum of Products
```

হিসেবে ভাবা যায়।

49. ADT এবং TypeScript

TypeScript এই ধারণাকে আরও পরিষ্কারভাবে express করতে পারে।

```
type Result<T> =  
  | {  
    type: "success";  
    data: T;  
  }
```

```
| {  
  type: "error";  
  message: string;  
};
```

তারপর:

```
function handle(result: Result<User>) {  
  if (result.type === "success") {  
    return result.data.name;  
  }  
  
  return result.message;  
}
```

JavaScript-এ runtime-এ এই pattern manually enforce করতে হয়।

50. ADT কেন গুরুত্বপূর্ণ?

কারণ এটি invalid state কমাতে সাহায্য করে।

খারাপ design:

```
{  
  loading: true,  
  data: users,  
  error: "Something went wrong"  
}
```

এখানে তিনটিই একসাথে থাকতে পারে।

কিন্তু ADT-style:

```
{  
  type: "loading"  
}
```

অথবা:

```
{  
  type: "success",  
  data: users  
}
```

অথবা:

```
{  
  type: "error",  
  message: "..."  
}
```

State space অনেক পরিষ্কার।

51. Maybe + Either + ADT

এখন সব concepts connect করি।

Maybe:

```
Some(value)  
None
```

এটাও এক ধরনের Sum Type।

Either:

```
Left(error)  
Right(value)
```

এটাও Sum Type।

Result:

```
Success(data)  
Failure(error)
```

এটাও Sum Type।

অর্থাৎ অনেক FP abstraction-এর নিচে ADT-এর চিত্তাধারা আছে।

52. Maybe এবং Optional Data

ধরি database থেকে:

```
findUserById(id)
```

User পাওয়া নাও যেতে পারে।

Instead of:

```
return null;
```

conceptually:

```
Some(User)
```

অথবা:

```
None
```

53. Either এবং Business Failure

ধরি:

```
transferMoney(from, to, amount)
```

Possible failures:

```
Invalid account  
Insufficient balance  
Account blocked  
Daily limit exceeded
```

Either model:

```
Left(error)
```

Success:

```
Right(transaction)
```

এতে failure explicitly data হিসেবে represent হয়।

54. Exception বনাম Either

Traditional:

```
try {  
  const user = createUser(data);  
} catch (error) {  
  ...  
}
```

Either style:

```
const result = createUser(data);  
  
if (result.type === "error") {  
  ...  
}
```

দুটিরই জায়গা আছে।

Either বিশেষভাবে useful যখন failure expected এবং business logic-এর অংশ।

Exception বেশি natural যখন:

- unexpected failure
 - infrastructure failure
 - programmer error
 - truly exceptional situation
-

55. `map` + `flatMap` একসাথে

ধরি:

```
const parseNumber = value => {  
  const number = Number(value);  
  
  if (Number.isNaN(number)) {  
    return Left("Invalid number");  
  }  
  
  return Right(number);  
};
```

এখন:

```
const double = value =>  
  value * 2;
```

Pipeline:

```
parseNumber("10")  
  .map(double);
```

Result:

```
Right(20)
```

কিন্তু যদি:

```
parseNumber("abc")
```

হয়:

```
Left("Invalid number")
```

`double` execute হবে না।

56. flatMap কখন?

যখন transformation function context-এর ভিতরের result return করে।

Normal:

```
x => y
```

তখন:

```
map
```

Context-aware:

```
x => Container(y)
```

তখন:

```
flatMap
```

57. Simple Comparison

```
map
```

```
Container<A>
```

```
↓
```

```
A → B
```

```
↓
```

```
Container<B>
```

```
flatMap
```

```
Container<A>
```

```
↓
```

```
A → Container<B>
```

↓
Container

এটাই সবচেয়ে important mental model ।

58. flatMap শুধু Monad-এর জন্য নয়

JavaScript Array-তেও:

```
const result = [  
  [1, 2],  
  [3, 4]  
].flatMap(x => x);
```

Result:

```
[1, 2, 3, 4]
```

এখানেও basic idea:

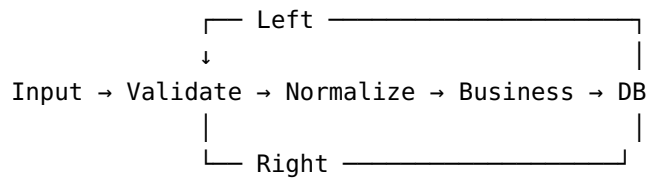
```
map + flatten
```

59. Functional Error Pipeline

একটি production-style conceptual pipeline:

```
Raw Input  
  ↓  
Validation  
  ↓  
Normalization  
  ↓  
Business Rules  
  ↓  
Database  
  ↓  
Response Mapping
```

Either ব্যবহার করলে:



একবার failure হলে error path ধরে শেষ পর্যন্ত যেতে পারে।

60. Lazy + Functional Pipeline

Large dataset processing:

```
File
↓
Generator
↓
map
↓
filter
↓
transform
↓
output
```

সব data memory-তে load করার প্রয়োজন নেই।

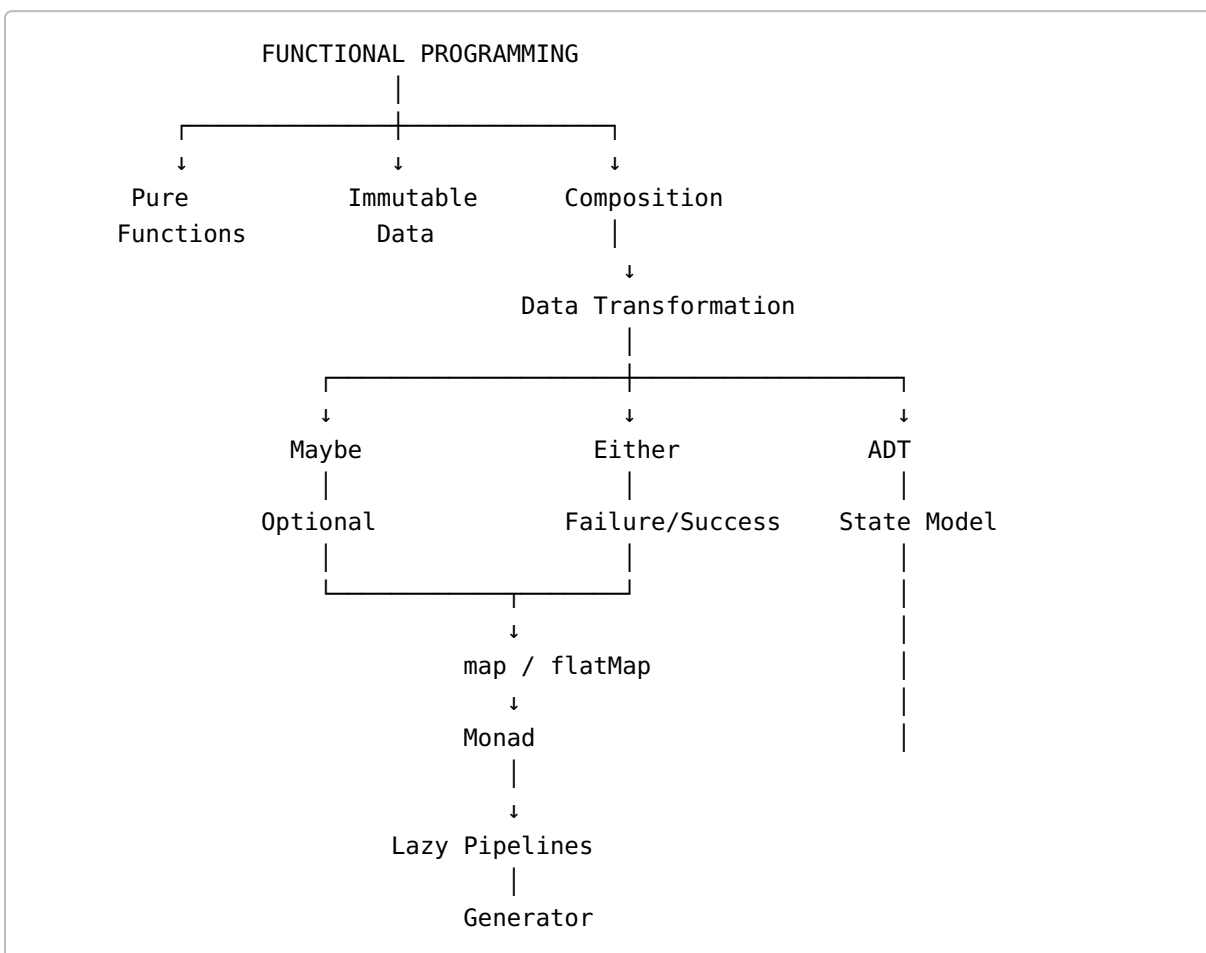
Node.js backend-এ এটি:

```
Readable Stream
↓
Transform
↓
Transform
↓
Writable
```

এর সাথে conceptually খুব related।

61. Functional Programming-এর একটি বড় Mental Model

পুরো section-টাকে এভাবে ভাবো:



62. Production Example: User Lookup

ধরি:

```
findUser(id)
```

User না-ও থাকতে পারে।

Conceptually:

Maybe<User>

তারপর:

```
Maybe<User>
  ↓
map(getProfile)
  ↓
Maybe<Profile>
  ↓
map(getAddress)
  ↓
Maybe<Address>
```

যদি কোনো জায়গায় data না থাকে:

None

63. Production Example: Order Processing

ধরি:

```
validateOrder
calculatePrice
checkInventory
chargePayment
createOrder
```

প্রতিটি business operation failure করতে পারে।

তাই:

Either<Error, Order>

conceptually useful।

Pipeline:

```
Input
↓
validateOrder
↓
calculatePrice
↓
checkInventory
↓
chargePayment
↓
createOrder
↓
Success
```

যেকোনো stage:

```
Left(error)
```

হলে pipeline failure track-এ যায়।

64. Functional Programming এবং React

React-এর state modeling-এ ADT খুব useful।

থারাপ:

```
const [loading, setLoading] = useState(false);
const [data, setData] = useState(null);
const [error, setError] = useState(null);
```

এখানে inconsistent state তৈরি হতে পারে।

যেমন:

```
loading = false
data = null
error = null
```

এটা ambiguous।

ADT style:

```
const state = {  
  type: "loading"  
};
```

Success:

```
const state = {  
  type: "success",  
  data  
};
```

Error:

```
const state = {  
  type: "error",  
  error  
};
```

এতে state model explicit হয়।

65. Functional Programming এবং Backend

Node.js backend-এ FP concepts বিশেষভাবে useful:

Validation

Input → validation → Either

Data transformation

DB result → map → API DTO

Optional data

Maybe/Option

Async pipeline

Promise → map/flatMap-like chaining

Streaming

Generator / Stream

State modeling

ADT / Result

66. Common Mistake: Monad মানেই Magic

Monad কোনো magical JavaScript object না।

Monad হলো একটি abstraction pattern।

মূল idea:

Context-এর ভিতরের computation
+
context-aware chaining

JavaScript-এ আমরা এর কাছাকাছি behavior দেখি:

Promise
Array
Maybe
Either

তবে সবগুলো একই semantic বা mathematical structure নয়—তাদের behavior এবং laws আলাদা।

67. Common Mistake: map আর flatMap একই

না।

```
map:  
A → B
```

কিন্তু:

```
flatMap:  
A → Container<B>
```

যখন function nested container তৈরি করছে তখন `flatMap` দরকার হয়।

68. Common Mistake: Functional Programming মানেই No Side Effect

Production application-এ side effect পুরোপুরি eliminate করা সম্ভব নয়।

Database:

```
INSERT
```

API:

```
fetch()
```

File:

```
writeFile()
```

সব side effect।

FP-এর লক্ষ্য:

Side effect-কে isolate এবং control করা।

যেমন:

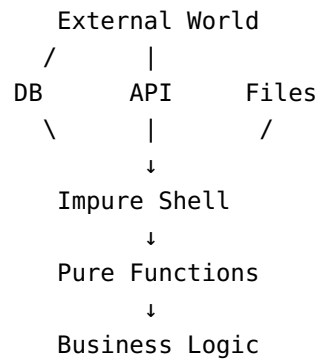
```
Pure Core  
↓
```

```
Business Logic
  ↓
Impure Shell
  ↓
DB / API / File / Network
```

এটি খুব powerful architecture ।

69. Pure Core, Impure Shell

একটি ভালো backend architecture:



Business logic যত বেশি pure করা যায়:

- testing সহজ
- debugging সহজ
- reasoning সহজ
- reuse সহজ
- deterministic behavior বেশি

70. Function Composition বনাম Inheritance

FP সাধারণত behavior combine করতে composition prefer করে ।

Inheritance:

```
UserService
  ↓
AdminUserService
```

↓
SuperAdminUserService

Composition:

```
validate  
+  
authorize  
+  
log  
+  
cache  
+  
execute
```

তারপর combine:

```
pipeline =  
validate  
→ authorize  
→ log  
→ execute
```

এতে coupling কমাতে পারে।

71. Functional Programming-এর গুরুত্বপূর্ণ Vocabulary

Term	সহজ অর্থ
Pure Function	একই input → একই output, side effect নেই
Immutability	Existing data mutate না করা
Referential Transparency	Expression safely result দিয়ে replace করা যায়
Higher-Order Function	Function নেয়/return করে
Composition	Function combine করা
Currying	Arguments একেক করে নেওয়া
Functor	<code>map</code> দিয়ে context-এর value transform
Monad	Context-aware computation chain

Term	সহজ অর্থ
Maybe	Optional value
Either	Success/failure
Lazy Evaluation	প্রয়োজন হলে computation
Generator	Lazy iterator তৈরি করে
Point-free	Explicit argument বাদ দিয়ে function composition
ADT	Data variants এবং combinations model করার pattern
Sum Type	A অথবা B
Product Type	A এবং B
Tagged Union	<code>type</code> /tag দিয়ে variant identify করা

72. Senior Developer Mental Model

এই section-এর সবচেয়ে গুরুত্বপূর্ণ mental model:

Maybe

```
Maybe<A>
```

মানে:

A থাকতে পারে, নাও থাকতে পারে।

Either

```
Either<E, A>
```

মানে:

Error অথবা successful value।

Functor

```
F<A>
  ↓ map
F<B>
```

মানে:

Container-এর ভিতরের value transform করে, structure রেখে দাও।

Monad

```
M<A>
  ↓ flatMap
A → M<B>
  ↓
M<B>
```

মানে:

এমন computation chain করো যেখানে প্রতিটি step একই context-এর result দিতে পারে।

Lazy Evaluation

```
Computation
  ↓
wait
  ↓
demand
  ↓
execute
```

মানে:

দরকার না হলে কাজ করো না।

Point-free

```
data
  ↓
f
  ↓
g
  ↓
h
```

মানে:

Data explicitly নাম না দিয়েও transformation pipeline তৈরি করা।

ADT

```
State =
  Loading
  | Success(Data)
  | Error(Error)
```

মানে:

Possible states-কে explicitভাবে model করা।

73. Quick Revision

01. Pure Function

```
x => x * 2
```

predictable।

02. Maybe

```
Some(value)
None
```

optional data।

03. Either

```
Left(error)  
Right(value)
```

explicit failure/success ।

04. Functor

```
map
```

container-এর ভিতরের value transform করে ।

05. Monad

```
flatMap
```

context-returning computation chain করে ।

06. Lazy Evaluation

```
Generator
```

demand অনুযায়ী computation করে ।

07. Point-free

```
compose(f, g)
```

explicit argument কমিয়ে composition ।

08. ADT

```
Sum Type  
+  
Product Type
```

data/state model করে ।

74. Interview Questions

Basic

Q1. Pure function কী?

যে function একই input-এর জন্য একই output দেয় এবং observable side effect করে না।

Q2. Immutability কেন useful?

Predictability, debugging, testing এবং state management সহজ করে।

Q3. Maybe কী solve করে?

Missing/optional value safely handle করে।

Q4. Either কী?

Success এবং failure explicit data হিসেবে model করে।

Intermediate

Q5. Functor কী?

এমন structure যেখানে `map` ব্যবহার করে ভিতরের value transform করা যায় structure না বদলে।

Q6. `map` এবং `flatMap` -এর difference কী?

```
map:  
A → B  
  
flatMap:  
A → M<B>
```

Q7. Generator lazy কেন?

`yield` execution pause করে এবং পরের value demand হলে computation resume হয়।

Q8. Point-free programming কী?

Function-এর argument explicitly mention না করে composition-এর মাধ্যমে logic express করা।

Advanced

Q9. Maybe এবং Either-এর difference?

```
Maybe:  
Some / None  
  
Either:  
Left(error) / Right(value)
```

Maybe missing value model করে, Either failure-এর কারণও carry করতে পারে।

Q10. Monad-এর practical use কী?

Context-aware computations chain করা এবং nested context flatten করা।

Q11. ADT-এর Sum এবং Product Type কী?

```
Sum:  
A OR B  
  
Product:  
A AND B
```

Q12. Railway-oriented programming কী?

Success এবং failure-এর দুইটি logical path ধরে data processing pipeline তৈরি করা।

75. Practice Tasks

Task 1 — Maybe

নিজে implement করো:

```
Maybe.of(value)  
Maybe.map(fn)  
Maybe.flatMap(fn)  
Maybe.getOrElse(defaultValue)
```

তারপর:

```
Maybe
  .of(user)
  .map(...)
  .map(...)
  .map(...)
```

দিয়ে nested property access করো।

Task 2 — Either

Implement:

```
Left
Right
```

এবং:

```
map()
flatMap()
fold()
```

তারপর একটি validation pipeline বানাও।

```
validateEmail
→ validatePassword
→ createUser
```

Task 3 — Lazy Pipeline

Generator দিয়ে implement করো:

```
lazyMap()
lazyFilter()
take()
```

তারপর:

```
infinite numbers
→ multiply
→ filter
→ take first 10
```

করো।

Task 4 — ADT

একটি API state model করো:

```
idle
loading
success
error
```

তারপর একটি `render()` function লেখো।

Task 5 — Functional Backend Pipeline

একটি order processing system design করো:

```
validate
→ normalize
→ calculate price
→ check inventory
→ payment
→ save
```

Expected model:

```
Either<Error, Order>
```

76. Final Senior-Level Takeaway

Functional Programming-এর এই পুরো section-কে ৬টা isolated topic হিসেবে দেখবে না।

বরং:

```
Pure Functions
  ↓
Immutability
  ↓
Composition
  ↓
Functor
  ↓
Monad
  ↓
Maybe / Either
  ↓
Safe Data Pipelines
```

আর data processing-এর ক্ষেত্রে:

```
Data
  ↓
Lazy Generator
  ↓
map
  ↓
filter
  ↓
flatMap
  ↓
result
```

আর state modeling-এর ক্ষেত্রে:

```
ADT
|
├── Product Type
|   ↓
|   A + B + C
└── Sum Type
    ↓
    A | B | C
```

সবচেয়ে গুরুত্বপূর্ণভাবে মনে রাখবে:

Maybe বলে "value থাকতে পারে বা নাও থাকতে পারে।"

Either বলে "success অথবা failure—failure হলে কারণটাও carry করে।"

Functor বলে "context-এর ভিতরের value map করে।"

Monad বলে "context-aware computation chain করে এবং nested context avoid করে।"

Generator lazy computation সম্ভব করে।

Point-free composition-এর মাধ্যমে data flow express করে।

ADT possible data/state-কে explicit model করে।

এই conceptsগুলো ভালোভাবে আয়ত্ত করলে JavaScript-এ শুধু syntax জানা নয়—**complex business logic** কীভাবে **predictable**, **composable** এবং **maintainable**ভাবে **design** করতে হয়, সেটার একটা অনেক বেশি senior-level understanding তৈরি হবে।